Production Integration Runbook

Setup, validation, deployment and operations

Prepared 19 October 2026 | Repository: prasad830105/DEMOGRAFY_INT | Main branch commit: 4229b1d

Before production work

Use this runbook to bring up a staging copy and prepare the integration boundary. It describes the current prototype, which reads OSM through Overpass and stores aggregate counts in Supabase. Do not use the demo RLS policies for customer production data without security approval.

Need	Current requirement
Repository	https://github.com/prasad830105/DEMOGRAFY_INT; app.py is the Streamlit entry point.
Runtime	Python version supported by the selected Streamlit deployment; install requirements.txt.
Database	Supabase Postgres project with PostgREST enabled and suburb/count tables configured.
Secrets	SUPABASE_URL and SUPABASE_KEY stored in local .env for development or platform Secrets for hosting.
External network	Outbound HTTPS to Supabase REST and public Overpass endpoints.

Local staging run

1. Clone the repository and create an isolated Python environment.
2. Install dependencies with: python -m pip install -r requirements.txt.
3. Copy .env.example to .env and add the staging Supabase URL and anon/publishable key. Never commit .env.
4. Start the app with: python -m streamlit run app.py.
5. Search for two suburbs, select them, compare, and record the displayed source, counts, area, density, scores and elapsed time.

The app can search the bundled suburbs_import.csv if Supabase is not configured. That fallback supports suburb selection only; live OSM counts still need internet access, and saving counts requires Supabase.

Supabase setup

Apply the SQL files in a staging Supabase project. Inspect each script and the project's existing schema first; the import and synchronization scripts change active suburb rows.

Order	Script or action	Purpose
1	supabase_schema.sql	Create amenity_counts and its initial demo policies if the table does not exist. Review policies before production.
2	supabase_suburbs.sql	Create/extend the searchable suburbs table and active-suburb read policy.
3	supabase_suburb_bounds.sql	Add area/bound fields and create suburbs_import staging table.
4	Import suburbs_import.csv into public.suburbs_import	Load the prepared SAL catalogue and its coordinate rectangles.
5	supabase_suburb_bounds_sync.sql	Deactivate existing catalogue rows and synchronize imported rows into public.suburbs.
6	supabase_amenity_counts_updated_at.sql	Add updated_at if the current amenity_counts table lacks it.
7	supabase_osm_policy.sql or supabase_osm_bbox_policy.sql	Allow current OSM count source writes for the prototype. Do not adopt demo policies unchanged in production.

Expected count row

The app upserts one row per selected suburb and category into public.amenity_counts. The current source is osm_overpass_bbox_green_recreation; category is Parks; place_count is the integer feature count; updated_at is the fetch time. The primary key is (suburb, category), so category/source version changes need a migration plan before adding alternate definitions.

For production, consider adding source/version to the uniqueness key or a separate results table with a stable SAL code, source version, boundary version, fetched_at, count and status. The current primary key overwrites the last result for a suburb/category and is not an audit history.

Quick SQL verification

Run these read-only checks in the Supabase SQL Editor after importing the suburb catalogue and after an app comparison:

```
select count(*) as active_suburbs from public.suburbs where is_active = true;
```

```
select suburb, category, place_count, source, updated_at
from public.amenity_counts
where source = 'osm_overpass_bbox_green_recreation'
order by updated_at desc;
```

Secret handling

- Local: .env is ignored by Git. Use only staging credentials on developer machines.
- Hosted: configure SUPABASE_URL and SUPABASE_KEY through the hosting provider's secret settings.
- Never commit a service-role key. The app currently speaks to Supabase REST with an anon key, so RLS is the security boundary for writes and reads.
- Before production, replace the open demo policies with least-privilege policies and decide how unauthenticated public app users are authorized to request and persist counts.

Validation plan

Scenario	Expected result
Valid pair with saved current source rows	Loads the cached database counts and shows Supabase as the source.
Valid pair with missing cache rows and live Overpass	Queries only selected suburb rectangles, deduplicates OSM type/ID, calculates counts and syncs both rows.
Refresh live counts	Bypasses Streamlit and Supabase count caches; issues fresh Overpass request and updates the Supabase rows.
Overpass unavailable but saved current OSM rows exist	Shows saved data with a clear warning that live retrieval failed.
No current saved rows and Overpass unavailable	Shows an actionable error and does not silently substitute unrelated sample data.
Supabase unavailable	Falls back to local suburb search; reports DB failures, and does not claim the live result was synced.
Unknown/ambiguous suburb or same suburb twice	Prompts for a valid choice and blocks an identical pair.
Zero features	Shows count 0; pair scores remain 0 when both densities are zero.
Repeated same OSM element in query output	Counts it once by OSM element type and ID.

Operational metrics to add

- Total compare request duration and stage durations for local cache, Supabase read, Overpass request, Supabase write and render.
- Cache hit rates separated into Streamlit cache and Supabase durable cache.
- Overpass endpoint attempted, response code, retry count, result count and timeout/error category; never log credentials.
- Supabase read/write failure rate and stale-row age at the moment of use.
- User-visible result metadata: source, fetched timestamp, boundary method/version and any fallback status.

Deployment and release steps

1. Choose a release commit and deploy it to a separate staging app from the GitHub repository.

2. Set SUPABASE_URL and SUPABASE_KEY in the hosting platform's secret manager. Do not include values in documentation, logs or Git.

3. Apply production-reviewed SQL migrations in a planned order. Verify RLS with both allowed and denied requests before connecting the app.

4. Run the validation scenarios above against staging, including one cold pair and one warm pair. Record timings and the data source displayed in the UI.

5. Test the app from the target website in an iframe. Confirm frame policy, viewport sizing, keyboard use, authentication behavior and mobile layout.

6. Set monitoring and alerts for app errors, database failures and sustained Overpass errors. Publish the deployment URL only after owners approve the acceptance evidence.

Rollback and incident response

If a release produces incorrect counts or elevated failures, roll back to the last approved app commit and disable the refresh path while preserving read-only display of verified saved rows where safe. If credentials may have been exposed, rotate them, review Supabase logs and policies, and remove secrets from the affected deployment. If an Overpass endpoint is unavailable, the app already tries multiple public endpoints, but these endpoints do not provide a production availability guarantee.

Release acceptance record

Evidence	Record before release
Approved scope	OSM source, feature tags, boundary method, freshness and score meaning
Security review	RLS policies, secret storage, write authorization and abuse controls
Data validation	Example suburb pair, source counts, density calculation and data freshness
Performance	Cold/warm p50 and p95 duration against approved objective
Deployment	Commit, environment, URL, iframe host and rollback reference
Operations	Monitoring owner, incident route, cache refresh and data correction procedure